

Chapitre 3

Rappels d'arithmétique dans l'ensemble des entiers relatifs

Ce chapitre a pour objectif de consolider les bases de l'arithmétique dans l'ensemble des entiers relatifs ($\mathbb{Z}$).

3.1 Division euclidienne, congruence

3.1.1 Motivation

En informatique et en sciences de l'ingénieur, les nombres entiers ne sont pas de simples compteurs. L'arithmétique modulaire, qui repose sur la division euclidienne et les congruences, est le moteur de la cryptographie moderne (comme les protocoles qui sécurisent nos communications), mais aussi des algorithmes de hachage et de la génération de nombres pseudo-aléatoires. Maîtriser ces concepts permet de comprendre comment les machines traitent, transforment et sécurisent l'information de manière cyclique.

3.1.2 Approche par problème

Imaginons un capteur dans un système industriel qui enregistre une mesure de température toutes les 7 heures. Si le capteur est activé un lundi à 00h00, quel jour de la semaine la 100^{ème} mesure sera-t-elle prise ?

Ce type de problème cyclique ne nécessite pas de calculer le temps total écoulé de manière absolue, mais plutôt de s'intéresser au *reste* de la division du temps total par la durée d'un cycle. C'est précisément l'objet de la division euclidienne et de la notion de congruence, qui permettent de simplifier les grands nombres en se concentrant uniquement sur leur comportement périodique.

3.1.3 Définitions et théorèmes

La division euclidienne

Théorème 3.1.1 (Division euclidienne dans $\mathbb{Z}$). Soit $a \in \mathbb{Z}$ et $b \in \mathbb{Z}^*$. Il existe un unique couple d'entiers $(q, r) \in \mathbb{Z} \times \mathbb{N}$ tel que :

$$a = bq + r \quad \text{avec} \quad 0 \leq r < |b|$$

Remarque 3.1.1. Dans cette écriture, a est appelé le **dividende**, b le **diviseur**, q le **quotient** et r le **reste**. Il est crucial de noter que le reste r est toujours **positif ou nul**, même si a ou b sont négatifs.

Congruences

La notion de congruence permet de formaliser mathématiquement ces phénomènes cycliques en travaillant directement sur les restes.

Définition 3.1.1 (Congruence modulo n). Soit $n \in \mathbb{N}^*$. Deux entiers relatifs a et b sont dits congrus modulo n si et seulement si n divise leur différence $a - b$. On note :

$$a \equiv b \pmod{n}$$

Ceci est équivalent à dire que a et b ont le même reste dans la division euclidienne par n .

Propriété 3.1.1 (Opérations sur les congruences). La relation de congruence est une relation d'équivalence compatible avec l'addition et la multiplication. Si $a \equiv b \pmod{n}$ et $c \equiv d \pmod{n}$, alors :

- $a + c \equiv b + d \pmod{n}$
- $ac \equiv bd \pmod{n}$
- Pour tout entier naturel k , $a^k \equiv b^k \pmod{n}$

3.1.4 Exemples de difficulté croissante

Exemple 3.1.1 (Division avec des entiers négatifs). Effectuer la division euclidienne de $a = -17$ par $b = 5$.

Correction : On cherche $q \in \mathbb{Z}$ et $r \in \mathbb{N}$ tels que $-17 = 5q + r$ avec $0 \leq r < 5$. Si l'on prend $q = -3$, on a $5 \times (-3) = -15$. Le reste serait de -2 , ce qui est impossible car le reste doit être positif. On doit donc descendre au multiple inférieur : on prend $q = -4$. On obtient : $-17 = 5 \times (-4) + 3$. Le quotient est donc $q = -4$ et le reste est $r = 3$.

Exemple 3.1.2 (Calcul modulaire sur de grandes puissances). Déterminer le reste de la division euclidienne de 3^{2026} par 7.

Correction : L'objectif est de trouver une puissance de 3 qui soit congrue à 1 ou -1 modulo 7. Calculons les premières puissances :

- $3^1 \equiv 3 \pmod{7}$
- $3^2 = 9 \equiv 2 \pmod{7}$
- $3^3 = 27 \equiv -1 \pmod{7}$

C'est ce qu'il nous faut ! Exprimons l'exposant 2026 en fonction de 3. La division euclidienne de 2026 par 3 donne : $2026 = 3 \times 675 + 1$. On peut donc écrire :

$$3^{2026} = 3^{3 \times 675 + 1} = (3^3)^{675} \times 3^1$$

En passant aux congruences modulo 7 :

$$3^{2026} \equiv (-1)^{675} \times 3 \pmod{7}$$

Comme 675 est impair, $(-1)^{675} = -1$.

$$3^{2026} \equiv -1 \times 3 \pmod{7} \equiv -3 \pmod{7}$$

Puisque le reste doit être positif, on ajoute 7 : $-3 + 7 = 4$.

$$3^{2026} \equiv 4 \pmod{7}$$

Le reste de la division euclidienne de 3^{2026} par 7 est donc 4.

3.1.5 Exercice pratique avec Python

Objectif : Implémenter la division euclidienne et vérifier des congruences.

En Python, les opérateurs `//` (quotient) et `%` (modulo) calculent la division euclidienne en respectant la règle mathématique du reste positif, même pour les entiers négatifs. C'est un avantage majeur par rapport à d'autres langages comme le C ou le Java.

```
def division_euclidienne(a, b):
    """Calcule et affiche le quotient et le reste de a par b."""
    if b == 0:
        return "Erreur : division par zéro"
    q = a // b
    r = a % b
    print(f"Dividende {a} = {b} * {q} + {r}")
    return q, r

def sont_congrus(a, b, n):
    """Vérifie si a et b sont congrus modulo n."""
    return (a % n) == (b % n)

# --- Tests ---
print("---- Division euclidienne ----")
division_euclidienne(17, 5)
```

```

division_euclidienne(-17, 5) # Python gère le reste positif !

print("\n--- Congruences ---")
# Vérifions le résultat de l'exemple 2 :  $3^{2026} = 4 \pmod{7}$ 
# En Python,  $3^{2026}$  calcule la puissance exacte instantanément.
resultat = sont_congrus(3**2026, 4, 7)
print(f" $3^{2026}$  est-il congru à 4 modulo 7 ? {resultat}")

```

3.2 PGCD et PPCM

3.2.1 Motivation

Dans de nombreux domaines de l'ingénierie et de l'informatique, nous sommes confrontés à des problèmes d'optimisation et de synchronisation. Comment diviser une surface de stockage en blocs carrés les plus grands possibles sans perte d'espace ? Comment déterminer le moment où deux processus exécutés en parallèle, avec des temps de cycle différents, vont se superposer ? Le PGCD (Plus Grand Commun Diviseur) et le PPCM (Plus Petit Commun Multiple) sont les outils mathématiques fondamentaux pour répondre à ces questions.

3.2.2 Approche par problème

Imaginons deux tâches dans un système d'exploitation embarqué. La tâche A s'exécute toutes les 15 millisecondes et la tâche B toutes les 25 millisecondes. Si elles démarrent exactement en même temps à $t = 0$, au bout de combien de temps s'exécuteront-elles à nouveau simultanément ?

Pour résoudre cela, on cherche un multiple commun à 15 et 25, et idéalement le plus petit possible pour anticiper le premier conflit : c'est le PPCM. À l'inverse, si l'on souhaite diviser une mémoire de 1500 octets et une autre de 2500 octets en segments de taille identique, la plus grande taille possible pour ces segments sera donnée par le PGCD.

3.2.3 Définitions et théorèmes

Le Plus Grand Commun Diviseur (PGCD)

Définition 3.2.1 (PGCD). *Soient a et b deux entiers relatifs non tous les deux nuls. L'ensemble des diviseurs communs à a et b est une partie finie et non vide de $\mathbb{Z}$. Le plus grand élément de cet ensemble est un entier naturel appelé Plus Grand Commun Diviseur de a et b . On le note $\text{PGCD}(a, b)$ ou simplement $a \wedge b$.*

Définition 3.2.2 (Nombres premiers entre eux). *Deux entiers relatifs a et b sont dits premiers entre eux si et seulement si leur seul diviseur commun*

positif est 1, c'est-à-dire :

$$\text{PGCD}(a, b) = 1$$

Le Plus Petit Commun Multiple (PPCM)

Définition 3.2.3 (PPCM). Soient a et b deux entiers relatifs non nuls. L'ensemble des multiples communs strictement positifs de a et b possède un plus petit élément. Cet entier naturel est appelé Plus Petit Commun Multiple de a et b . On le note $\text{PPCM}(a, b)$ ou simplement $a \vee b$.

Relation fondamentale

Il existe un lien analytique très fort entre le PGCD et le PPCM de deux nombres.

Théorème 3.2.1 (Lien entre PGCD et PPCM). Pour tous entiers relatifs non nuls a et b , le produit de leur PGCD et de leur PPCM est égal à la valeur absolue de leur produit :

$$\text{PGCD}(a, b) \times \text{PPCM}(a, b) = |ab|$$

3.2.4 Exemples de difficulté croissante

Exemple 3.2.1 (Calcul par décomposition en facteurs premiers). Déterminer le PGCD et le PPCM de $a = 60$ et $b = 84$.

Correction : On décompose d'abord en produit de facteurs premiers.
 $60 = 2^2 \times 3^1 \times 5^1$ $84 = 2^2 \times 3^1 \times 7^1$

— Pour le **PGCD**, on prend les facteurs premiers *communs* avec la *plus petite* puissance :

$$\text{PGCD}(60, 84) = 2^2 \times 3^1 = 12$$

— Pour le **PPCM**, on prend *tous* les facteurs premiers présents avec la *plus grande* puissance :

$$\text{PPCM}(60, 84) = 2^2 \times 3^1 \times 5^1 \times 7^1 = 420$$

On peut vérifier le théorème : $12 \times 420 = 5040$ et $60 \times 84 = 5040$.

Exemple 3.2.2 (Démonstration algébrique). Montrer que pour tout entier naturel n , les nombres $a = n$ et $b = n + 1$ sont premiers entre eux.

Correction : Soit d un diviseur commun positif à n et $n + 1$. Par définition, il existe des entiers k_1 et k_2 tels que $n = d \cdot k_1$ et $n + 1 = d \cdot k_2$.

Si d divise n et $n + 1$, il divise aussi toute combinaison linéaire de ces deux nombres, et en particulier leur différence :

$$(n + 1) - n = d \cdot k_2 - d \cdot k_1 = d(k_2 - k_1)$$

Donc $1 = d(k_2 - k_1)$. Le seul entier naturel qui divise 1 est 1. Donc $d = 1$. Leur seul diviseur commun étant 1, on a bien $\text{PGCD}(n, n + 1) = 1$. Ils sont premiers entre eux.

3.2.5 Exercice pratique avec Python

Objectif : Utiliser la bibliothèque `math` de Python pour calculer le PGCD et déduire le PPCM.

Depuis Python 3.9, les fonctions `gcd` (Greatest Common Divisor) et `lcm` (Least Common Multiple) sont directement incluses dans le module mathématique standard.

```
import math

def calculer_pgcd_ppcm(a, b):
    """Affiche le PGCD et le PPCM de deux entiers."""
    if a == 0 and b == 0:
        return "Le PGCD de 0 et 0 n'est pas défini."

    # Utilisation des fonctions natives de Python
    pgcd_val = math.gcd(a, b)

    # Si on utilise une version de Python < 3.9, on peut déduire le PPCM :
    # ppcm_val = abs(a * b) // pgcd_val
    # Sinon, on utilise math.lcm :
    ppcm_val = math.lcm(a, b)

    print(f"Pour a={a} et b={b} :")
    print(f" -> PGCD = {pgcd_val}")
    print(f" -> PPCM = {ppcm_val}")
    print(f" -> Vérification PGCD * PPCM = {pgcd_val * ppcm_val} (|ab| = {abs(a*b)})")

# --- Tests ---
calculer_pgcd_ppcm(60, 84)
print("-" * 20)
calculer_pgcd_ppcm(15, 25) # Retour à notre problème de synchronisation
```

3.3 Théorème de Gauss, Identité de Bézout, Algorithme d'Euclide

3.3.1 Motivation

La théorie des nombres ne se contente pas d'étudier les propriétés abstraites des entiers ; elle fournit les algorithmes qui sécurisent le monde numérique. L'algorithme d'Euclide est l'un des plus vieux algorithmes au monde, et sa version "étendue" (liée à l'identité de Bézout) permet de calculer des inverses modulaires. Ces calculs sont le cur mathématique du chiffrement RSA, utilisé quotidiennement pour sécuriser les transactions bancaires et les communications sur Internet.

3.3.2 Approche par problème

Imaginez que vous disposiez d'un récipient de 5 litres et d'un autre de 3 litres, sans aucune graduation. Comment faire pour obtenir exactement 1 litre d'eau ?

Mathématiquement, cela revient à chercher deux entiers u et v (le nombre de fois où l'on remplit ou vide chaque récipient) tels que $5u + 3v = 1$. Cette équation, appelée équation diophantienne linéaire, possède des solutions entières uniquement grâce aux relations profondes qui lient le nombre 5, le nombre 3, et leur PGCD. Le théorème de Bézout et l'algorithme d'Euclide nous donnent la méthode systématique pour trouver ces solutions.

3.3.3 Définitions et théorèmes

L'Algorithme d'Euclide

L'algorithme d'Euclide permet de calculer le PGCD de deux entiers sans avoir à chercher tous leurs diviseurs. Il repose sur le lemme suivant : si $a = bq + r$, alors $\text{PGCD}(a, b) = \text{PGCD}(b, r)$.

Théorème 3.3.1 (Algorithme d'Euclide). *Soient a et b deux entiers naturels non nuls avec $a > b$. En effectuant des divisions euclidiennes successives :*

$$a = bq_1 + r_1 \quad (0 < r_1 < b)$$

$$b = r_1q_2 + r_2 \quad (0 < r_2 < r_1)$$

$$r_1 = r_2q_3 + r_3 \quad (0 < r_3 < r_2)$$

...

$$r_{n-2} = r_{n-1}q_n + r_n \quad (0 < r_n < r_{n-1})$$

$$r_{n-1} = r_nq_{n+1} + 0$$

Le dernier reste non nul, r_n , est exactement le PGCD(a, b).

[Image of flowchart of the Euclidean algorithm]

Identité de Bézout

Théorème 3.3.2 (Identité de Bézout). *Deux entiers relatifs a et b sont premiers entre eux si et seulement s'il existe deux entiers relatifs u et v tels que :*

$$au + bv = 1$$

Propriété 3.3.1 (Généralisation - Théorème de Bachet-Bézout). *Soient a et b deux entiers non nuls. Si $d = \text{PGCD}(a, b)$, alors il existe deux entiers relatifs u et v tels que :*

$$au + bv = d$$

De plus, l'équation $ax + by = c$ admet des solutions entières (x, y) si et seulement si c est un multiple de d .

Théorème de Gauss

Théorème 3.3.3 (Théorème de Gauss). *Soient a , b et c trois entiers relatifs non nuls. Si a divise le produit bc et si a et b sont premiers entre eux ($\text{PGCD}(a, b) = 1$), alors a divise c .*

3.3.4 Exemples de difficulté croissante

Exemple 3.3.1 (Recherche de PGCD par l'algorithme d'Euclide). *Déterminer le PGCD de 420 et 135.*

Correction : On applique l'algorithme d'Euclide par divisions successives :

$$— 420 = 135 \times 3 + 15 \text{ (le reste est 15)}$$

$$— 135 = 15 \times 9 + 0 \text{ (le reste est 0)}$$

Le dernier reste non nul est 15. Donc, $\text{PGCD}(420, 135) = 15$.

Exemple 3.3.2 (Résolution d'une équation avec Bézout et Gauss). *Résoudre dans $\mathbb{Z}^2$ l'équation (E) : $5x - 3y = 2$.*

Correction : 1. **Recherche d'une solution particulière :** On remarque de manière évidente que $5 \times 1 - 3 \times 1 = 2$. Donc le couple $(1, 1)$ est une solution particulière. 2. **Soustraction :** Soit (x, y) une solution. On a :

$$5x - 3y = 2$$

$$5(1) - 3(1) = 2$$

En soustrayant ligne à ligne : $5(x - 1) - 3(y - 1) = 0$, soit $5(x - 1) = 3(y - 1)$. 3. **Application du théorème de Gauss :** 5 divise $3(y - 1)$. Or $\text{PGCD}(5, 3) = 1$. D'après le théorème de Gauss, 5 divise $(y - 1)$. Il existe donc un entier k tel que $y - 1 = 5k$, d'où $y = 5k + 1$. 4. **Substitution :** On remplace $y - 1$ par $5k$ dans l'égalité $5(x - 1) = 3(y - 1)$:

$$5(x - 1) = 3(5k) \implies x - 1 = 3k \implies x = 3k + 1$$

Les solutions sont donc les couples de la forme $(3k + 1, 5k + 1)$ avec $k \in \mathbb{Z}$.

3.3.5 Exercice pratique avec Python

Objectif : Implémenter l'algorithme d'Euclide étendu.

Cet algorithme ne se contente pas de trouver le PGCD, il "remonte" les calculs pour trouver les coefficients u et v de l'identité de Bézout ($au + bv = \text{PGCD}$). C'est indispensable pour calculer des clés cryptographiques.

```
def euclide_etendu(a, b):
    """
    Retourne (pgcd, u, v) tels que  $a*u + b*v = \text{pgcd}$ 
    basé sur l'algorithme d'Euclide étendu.
    """
    if b == 0:
        return a, 1, 0
    else:
        # Appel récursif
        d, u_prime, v_prime = euclide_etendu(b, a % b)

        # On reconstitue les coefficients u et v
        u = v_prime
        v = u_prime - (a // b) * v_prime

    return d, u, v

# --- Tests ---
a, b = 420, 135
pgcd, u, v = euclide_etendu(a, b)

print(f"Pour a={a} et b={b} :")
print(f"PGCD trouvé : {pgcd}")
print(f"Coefficients de Bézout : u={u}, v={v}")
print(f"Vérification : {a}*({u}) + {b}*({v}) = {a*u + b*v}")

# Test avec le problème d'introduction (récipients de 5L et 3L)
d, u, v = euclide_etendu(5, 3)
print("\nProblème des récipients (5L et 3L) :")
print(f"5*({u}) + 3*({v}) = {d}")
# Interprétation : Remplir 2 fois le récipient de 5L (10L),
# et vider 3 fois son contenu dans celui de 3L (-9L) laisse 1L.
```

Travaux Dirigés : Arithmétique dans $\mathbb{Z}$

Exercice 1 : Restitution et définitions (*Taxonomie : Connaissance / Difficulté : Facile*)

1. Énoncer avec précision le théorème de la division euclidienne dans $\mathbb{Z}$, en insistant sur la condition portant sur le reste.
2. Soit $n \in \mathbb{N}^*$. Donner la définition mathématique exacte de la proposition : « a et b sont congrus modulo n ».
3. Rappeler l'énoncé du théorème de Gauss.

Exercice 2 : Propriétés fondamentales (*Taxonomie : Compréhension / Difficulté : Facile*)

L'objectif de cet exercice est de démontrer des propriétés du cours à partir des définitions.

1. Soient $a, b, c, d \in \mathbb{Z}$ et $n \in \mathbb{N}^*$. Démontrer que si $a \equiv b \pmod{n}$ et $c \equiv d \pmod{n}$, alors $a + c \equiv b + d \pmod{n}$.
2. Démontrer que si $a \equiv b \pmod{n}$, alors pour tout entier naturel k , $a^k \equiv b^k \pmod{n}$.

Exercice 3 : PGCD et littéral (*Taxonomie : Application / Difficulté : Moyenne*)

Soit n un entier naturel. On considère les entiers $A = n$ et $B = n + 1$.

1. En utilisant la définition des diviseurs communs, démontrer que $\text{PGCD}(A, B) = 1$.
2. En déduire, en appliquant l'identité de Bézout, que pour tout $n \in \mathbb{N}$, les nombres $2n + 1$ et $n(n + 1)$ sont premiers entre eux.

Exercice 4 : Réduction de PGCD (*Taxonomie : Analyse / Difficulté : Moyenne*)

Soient a et b deux entiers relatifs non nuls. On pose $d = \text{PGCD}(a, b)$.

1. Justifier l'existence de deux entiers a' et b' tels que $a = da'$ et $b = db'$.
2. Analyser la relation entre a' et b' en démontrant rigoureusement que $\text{PGCD}(a', b') = 1$. (*Indication : Utiliser le théorème de Bachet-Bézout*).

Exercice 5 : Conséquences du théorème de Gauss (*Taxonomie : Synthèse / Difficulté : Difficile*)

Soient a, b et c trois entiers relatifs non nuls. L'objectif est de démontrer une propriété de divisibilité simultanée.

1. On suppose que a divise c , que b divise c , et que $\text{PGCD}(a, b) = 1$. Construire une démonstration prouvant que le produit ab divise c . (*Indication : Commencer par écrire $c = ak = bl$, puis utiliser le théorème de Gauss*).
2. Montrer par un contre-exemple que la condition $\text{PGCD}(a, b) = 1$ est strictement nécessaire.

Exercice 6 : Évaluation de divisibilité (*Taxonomie : Évaluation / Difficulté : Difficile*)

On considère l'expression $E(n) = n(n+1)(2n+1)$ pour tout entier naturel n .

1. Évaluer la divisibilité de $E(n)$ par 2 en distinguant les cas de parité de n .
2. Évaluer la divisibilité de $E(n)$ par 3 en utilisant les congruences modulo 3.
3. En déduire, à l'aide du résultat de l'Exercice 5, que $E(n)$ est toujours un multiple de 6.

Exercice 7 : Somme et produit d'entiers premiers entre eux (*Taxonomie : Synthèse / Difficulté : Très Difficile*)

Soient a et b deux entiers relatifs non nuls tels que $\text{PGCD}(a, b) = 1$.

1. Démontrer que $\text{PGCD}(a+b, a) = 1$ et que $\text{PGCD}(a+b, b) = 1$.
2. En déduire rigoureusement que $\text{PGCD}(a+b, ab) = 1$.
3. Cette propriété est-elle conservée pour la somme des carrés ? Démontrer que $\text{PGCD}(a^2 + b^2, ab) = 1$.

Exercice 8 : Construction du système des restes chinois (cas $n=2$) (*Taxonomie : Création / Difficulté : Très Difficile*)

Soient p et q deux entiers naturels non nuls et premiers entre eux ($\text{PGCD}(p, q) = 1$). On s'intéresse au système de congruences suivant, où a et b sont des entiers quelconques :

$$(S) \begin{cases} x \equiv a \pmod{p} \\ x \equiv b \pmod{q} \end{cases}$$

1. Justifier l'existence de deux entiers u et v tels que $pu + qv = 1$.
2. Créer une solution particulière x_0 au système (S) en fonction de p, q, u, v, a et b . Démontrer que cette valeur x_0 satisfait bien les deux équations du système.
3. Formuler et démontrer la forme générale de toutes les solutions x du système (S) . Sur quel modulo l'unicité de la solution est-elle garantie ?

Travaux Pratiques (TP) : Implémentations Algorithmiques

L'objectif de cette séance de Travaux Pratiques est de traduire les théorèmes d'arithmétique dans l'ensemble $\mathbb{Z}$ en algorithmes fonctionnels. Les implémentations pourront être réalisées en langage Python.

Exercice 1 : La division euclidienne *from scratch*

1. [Bloom : Compréhension | Difficulté : Facile]

Écrire une fonction `division_naïve(a, b)` qui prend deux entiers naturels a et b ($b > 0$) et qui retourne le quotient q et le reste r en utilisant uniquement des soustractions successives (sans utiliser les opérateurs `//` et `%`).

2. [Bloom : Application | Difficulté : Moyenne]

Adapter cette fonction pour qu'elle respecte le théorème mathématique de la division euclidienne dans $\mathbb{Z}$, c'est-à-dire en gérant correctement les cas où a et/ou b sont des entiers strictement négatifs (rappel : le reste r doit toujours vérifier $0 \leq r < |b|$).

Exercice 2 : Congruences et tests de primalité

1. [Bloom : Application | Difficulté : Facile]

Écrire une fonction `est_premier(n)` qui utilise l'opérateur modulo pour déterminer si un entier $n \geq 2$ est un nombre premier. L'algorithme devra s'arrêter intelligemment à $\lfloor \sqrt{n} \rfloor$ pour optimiser le calcul.

2. [Bloom : Analyse | Difficulté : Moyenne]

Implémenter une fonction `crible_eratosthene(N)` qui retourne la liste de tous les nombres premiers inférieurs ou égaux à N , en appliquant le principe d'élimination des multiples.

Exercice 3 : Les deux visages du PGCD

1. [Bloom : Application | Difficulté : Facile]

Implémenter une fonction `pgcd_soustraction(a, b)` basée sur l'algorithme original d'Euclide utilisant les soustractions successives.

2. [Bloom : Application | Difficulté : Moyenne]

Implémenter une fonction `pgcd_modulo(a, b)` utilisant les divisions euclidiennes successives (opérateur `%`).

3. [Bloom : Évaluation | Difficulté : Moyenne]

Écrire un script qui compare le temps d'exécution de ces deux fonctions pour $a = 123456789$ et $b = 123$. Conclure sur la complexité algorithmique.

Exercice 4 : Calcul du PPCM optimisé

1. [Bloom : Compréhension | Difficulté : Facile]

Déduire du théorème vu en cours une fonction `ppcm(a, b)` qui calcule le Plus Petit Commun Multiple de deux entiers en faisant appel à la fonction `pgcd_modulo(a, b)`.

2. [Bloom : Synthèse | Difficulté : Moyenne]

Écrire une fonction `ppcm_liste(L)` qui prend en paramètre une liste de n entiers relatifs et qui retourne le PPCM global de tous les éléments de cette liste.

Exercice 5 : L'Algorithme d'Euclide Étendu

1. [Bloom : Analyse | Difficulté : Difficile]

Implémenter la fonction `euclide_etendu(a, b)` qui retourne un triplet (d, u, v) tel que $d = \text{PGCD}(a, b)$ et $au + bv = d$. Vous pouvez utiliser une approche récursive ou itérative au choix.

2. [Bloom : Évaluation | Difficulté : Moyenne]

Tester la fonction avec $a = 240$ et $b = 46$, puis afficher la chaîne de caractères vérifiant l'équation de Bézout correspondante.

Exercice 6 : Résolution algorithmique d'équations diophantiennes

1. [Bloom : Synthèse | Difficulté : Difficile]

Écrire une fonction `resoudre_diophantienne(a, b, c)` qui cherche à résoudre l'équation $ax + by = c$. La fonction devra vérifier si une solution existe grâce au théorème de Bachet-Bézout, et si oui, utiliser la fonction `euclide_etendu` pour retourner une solution particulière (x_0, y_0) .

2. [Bloom : Création | Difficulté : Difficile]

Enrichir la fonction pour qu'elle prenne en compte deux bornes $X_{\min}$ et $X_{\max}$, et qu'elle retourne la liste de toutes les solutions (x, y) telles que $X_{\min} \leq x \leq X_{\max}$.

Exercice 7 : Application Cryptographique (Chiffrement Affine)

1. [Bloom : Application | Difficulté : Moyenne]

Le chiffrement affine transforme une lettre en lui associant un rang x (de 0 pour A à 25 pour Z), puis calcule un nouveau rang $y \equiv ax + b \pmod{26}$. Écrire une fonction `chiffrement_affine(texte, a, b)` qui retourne le texte chiffré. Que doit vérifier la clé a pour que le déchiffrement soit possible ?

2. [Bloom : Synthèse | Difficulté : Difficile]

Pour déchiffrer, il faut trouver l'inverse modulaire de a modulo 26,

c'est-à-dire un entier a' tel que $a \times a' \equiv 1 \pmod{26}$. Utiliser l'algorithme d'Euclide étendu pour écrire une fonction `inverse_modulaire(a, n)` et en déduire la fonction `dechiffrement_affine(texte_chiffre, a, b)`.

Exercice 8 : Introduction à RSA (Génération de clés)

1. [Bloom : Création | Difficulté : Très Difficile]

L'algorithme de cryptographie asymétrique RSA repose fortement sur l'arithmétique dans $\mathbb{Z}$. Écrire une fonction globale `generer_cles_rsa(p, q)` où p et q sont deux nombres premiers distincts donnés en paramètre. L'algorithme devra :

- Calculer le module $n = p \times q$.
- Calculer l'indicatrice d'Euler $\phi(n) = (p - 1)(q - 1)$.
- Choisir un entier e tel que $1 < e < \phi(n)$ et $\text{PGCD}(e, \phi(n)) = 1$.
- Calculer d , l'inverse modulaire de e modulo $\phi(n)$ (à l'aide de l'exercice 7).
- Retourner la clé publique (n, e) et la clé privée (n, d) .

Aller plus loin : L'arithmétique au cur de la Data Science

L'arithmétique dans $\mathbb{Z}$, bien qu'étudiée depuis l'Antiquité, est aujourd'hui l'un des piliers de l'ingénierie des données (Data Science) et de l'intelligence artificielle. Au-delà de la cryptographie (RSA) que nous avons évoquée, les concepts de division euclidienne et de congruence sont indispensables pour optimiser la mémoire et les temps de calcul dans les algorithmes d'apprentissage automatique (Machine Learning).

Le défi des données textuelles en NLP

Dans le domaine du traitement du langage naturel (NLP), les algorithmes ne comprennent pas les mots : ils ont besoin de nombres. Pour entraîner un modèle de classification de textes ou analyser des séquences de mots (ce qu'on appelle des *N-grammes*), on transforme généralement le vocabulaire en un immense tableau (un dictionnaire).

Cependant, sur un corpus de texte réel (comme l'ensemble des articles Wikipédia), le nombre de mots uniques et de combinaisons de *N-grammes* peut atteindre des dizaines de millions. Créer une matrice où chaque colonne représente un *N-gramme* spécifique demande une quantité de mémoire RAM gigantesque, souvent indisponible sur des machines standards. Comment alors représenter ces données spatialement sans saturer la mémoire ?

La solution : Le **n** Feature Hashing **z** (L'astuce du hachage)

C'est ici qu'intervient l'arithmétique modulaire à travers une technique appelée le **Feature Hashing** (ou Hashing Trick). Plutôt que de maintenir un dictionnaire exact, on utilise une fonction mathématique (une fonction de hachage) qui transforme chaque chaîne de caractères en un grand nombre entier abstrait.

Ensuite, pour forcer ces grands nombres à rentrer dans un espace mémoire de taille fixe et contrôlée (par exemple M colonnes), on applique une simple congruence modulo M :

$$\text{Index du vecteur} = \text{Fonction_Hachage}(\text{mot}) \pmod{M}$$

Exemple conceptuel : Supposons que nous fixions la taille de notre mémoire à $M = 1000$ dimensions. Si la fonction de hachage convertit le bi-gramme *"intelligence artificielle"* en l'entier 4598273, sa position dans notre vecteur mémoire sera simplement le reste de sa division euclidienne par 1000 :

$$4598273 \equiv 273 \pmod{1000}$$

Le modèle stockera l'information de ce bi-gramme à l'index 273.

Gestion des collisions et propriétés statistiques

Que se passe-t-il si un autre mot possède également un reste de 273 (ce qu'on appelle une collision) ? Mathématiquement, le modèle va additionner les fréquences de ces deux mots au même endroit.

Bien que cela semble être une perte d'information (deux mots différents mélangés), les mathématiques statistiques démontrent que si M est suffisamment grand (et souvent choisi comme un nombre premier pour de meilleures propriétés de répartition arithmétique), l'impact de ces collisions sur les performances du modèle de Machine Learning est négligeable.

Ainsi, grâce à la théorie des congruences, des modèles très complexes peuvent s'entraîner sur des flux de données continus et infinis avec une empreinte mémoire fixe et strictement maîtrisée en temps constant $\mathcal{O}(1)$.

*Pour aller plus loin sur ce sujet, vous pouvez vous documenter sur l'algorithme **MurmurHash**, très utilisé dans les bibliothèques de Machine Learning comme Scikit-Learn (via **FeatureHasher**), ou sur la manière dont les architectures de plongement lexical (Word Embeddings) comme **GloVe** gèrent les matrices de co-occurrence massives.*

Projet d'Application : Le **Hashing Trick** en Analyse de Données

Contexte du Projet

Imaginez que vous avez mis en place un système de *web scraping* pour récupérer en direct les commentaires textuels de milliers de matchs de football (ex : **Tir cadré de l'attaquant**, **Faute dangereuse au milieu du terrain**, etc.). Votre objectif est d'entraîner un modèle de Machine Learning capable d'analyser ces flux de textes pour calculer des statistiques en temps réel ou prédire l'issue d'une action.

Le problème : le vocabulaire total généré par ces milliers de matchs est gigantesque. Construire un dictionnaire classique en mémoire vive (RAM) pour associer chaque mot ou N-gramme à un index unique va saturer vos serveurs.

Vous allez donc utiliser l'arithmétique dans $\mathbb{Z}$, et plus précisément la division euclidienne, pour implémenter de zéro la technique du **Feature Hashing** étudiée dans la section précédente.

Objectifs

1. Implémenter une vectorisation de texte classique (avec dictionnaire) pour en montrer les limites.
2. Implémenter une fonction de *Feature Hashing* basée sur l'opération modulo.
3. Analyser mathématiquement le taux de collision en fonction de la taille de l'espace mémoire M .

Cahier des charges et Travail demandé

Étape 1 : Simulation des données textuelles

Pour simuler notre flux de données, créez un script Python qui génère une liste de phrases ou récupérez un corpus de texte volumineux (par exemple, un fichier texte contenant un grand nombre de commentaires sportifs ou d'articles). Développez une fonction `extraire_mots(texte)` qui nettoie le texte (minuscules, ponctuation) et retourne une liste de mots.

Étape 2 : L'approche classique (Dictionnaire exact)

- Écrivez une fonction `vectoriseur_classique(corpus)` qui :
- Parcourt tous les mots du corpus.
 - Assigne un identifiant unique (un entier de 0 à $V - 1$, où V est la taille du vocabulaire) à chaque nouveau mot rencontré en utilisant un dictionnaire Python (`dict`).

- **Question :** Quelle est la complexité spatiale de cette approche si le flux de mots est infini ?

Étape 3 : L'approche arithmétique (Feature Hashing)

Ici, nous n'utilisons plus de dictionnaire pour mémoriser les mots. Nous fixons une taille de mémoire maximale, par exemple $M = 1024$ colonnes.

- Écrivez une fonction `vectoriseur_hachage(corpus, M)` qui :
- Utilise une fonction de hachage standard pour convertir chaque mot en un entier abstrait. Vous pouvez utiliser la fonction native `hash()` de Python ou importer `hashlib`.
 - Calcule l'index du mot dans le vecteur mémoire en utilisant la congruence :

$$\text{Index} \equiv \text{hash}(\text{mot}) \pmod{M}$$

- Construit un vecteur de taille M comptant la fréquence d'apparition des index.

Étape 4 : Analyse arithmétique des collisions

Une collision survient lorsque deux mots différents m_1 et m_2 vérifient :

$$\text{hash}(m_1) \equiv \text{hash}(m_2) \pmod{M}$$

1. Modifiez votre fonction de hachage pour qu'elle compte le nombre exact de mots différents qui "tombent" sur le même index.
2. Testez votre algorithme avec $M = 100$, $M = 1009$ (un nombre premier) et $M = 10000$.
3. Tracez un graphique (avec `matplotlib`) représentant le nombre de collisions en fonction de la taille M .
4. **Analyse :** Pourquoi recommande-t-on souvent d'utiliser un nombre premier pour la dimension M en Data Science ? (Faites le lien avec les diviseurs communs vus dans le Chapitre III).

Bonus (Pour aller très loin)

Remplacez les mots simples par des bi-grammes (paires de mots adjacents). Comparez l'explosion de la mémoire RAM avec l'approche classique par rapport à la stabilité absolue de la mémoire avec votre approche par modulo.